

CODEFEST AD ASTRA 2026

Documento tecnico — Etapa 1: Base de Conocimiento Vectorial

1. El corpus y su preprocesamiento

El corpus entregado por ADL son **1826 documentos** (3 GB) de 21 observatorios repartidos en los tres fenomenos: 760 PDF, 964 JSON, 26 CSV, 73 PBF, 8 imagenes, 6 XLSX y 1 TXT. La heterogeneidad es el primer problema de ingenieria del reto: un mismo indice tiene que absorber informes de 400 paginas de SIPRI o del AI Index, articulos web serializados en JSON, datasets bibliograficos de decenas de MB y teselas de mapas vectoriales. De los 1826 documentos, **1818 aportan texto**; los 8 restantes son 5 imagenes sin texto legible, un JSON vacio y dos archivos con extension *.pdf* que en realidad son paginas HTML de error de la descarga.

Cada formato de origen tiene un extractor dedicado que produce un objeto comun (*RawDocument*) con el texto crudo y metadata auxiliar: **PDF** (pypdfium2, preservando el orden de lectura por pagina y con deteccion heuristica de boilerplate — lineas identicas repetidas en $\geq 30\%$ de las paginas de un documento de 3 o mas paginas se descartan como cabecera/pie de pagina), **HTML** (trafilatura, que separa el contenido principal del menu de navegacion, anuncios y pie de pagina, y preserva encabezados como senales Markdown), **JSON** (mapeo explicito de campos conocidos como *title/body_paragraphs/body_text*, con fallback generico para esquemas no anticipados), **CSV/XLSX** (cada fila se convierte en pares *columna: valor* y se trata como unidad atomica de fragmentacion) e **imagenes** (OCR opcional via pytesseract, con degradacion controlada si el binario de tesseract no esta disponible). Tres decisiones merecen justificacion explicita porque las impuso el corpus real:

- **pypdfium2 en vez de pdfplumber** para los PDF. Con 760 PDF y del orden de 30.000 paginas, el costo de extraccion deja de ser irrelevante: medido sobre este corpus, pypdfium2 lee unas 3.000 paginas en 4,5 s, mientras que pdfplumber es dos ordenes de magnitud mas lento y convierte la fase offline de minutos en horas. Para texto plano de informes la calidad extraida es equivalente.
- **OCR selectivo.** Cerca del 5% de los PDF — casi todos los informes de riesgo de la Defensoria del Pueblo — son escaneos sin capa de texto. Cuando un documento rinde menos de 200 caracteres por pagina se asume escaneo y se rasteriza para pasarlo por tesseract, con un tope de 60 paginas por documento: sin ese tope un puñado de escaneos largos domina el tiempo total de la corrida.
- **Los .pbf no son OpenStreetMap.** Las 73 teselas de Amazon Underworld son *Mapbox Vector Tiles*, no el formato Protocol Buffer de OSM, asi que las librerias habituales (pyosmium, pyosrm) no las leen. Se decodifican con *mapbox-vector-tile*, se recorren las capas leyendo los atributos de cada elemento como pares *atributo: valor*, y se deduplican los elementos repetidos entre niveles de zoom, tal como indica la sec. 2.1.

Los CSV/XLSX llevan un tope de 500 filas por archivo: los datasets del AI Index son volcados bibliograficos de PubMed de hasta 35 MB, ajenos a las consultas del reto, y sin tope aportarian mas de 100.000 fragmentos de ruido que dominarian el espacio vectorial. Con tope el documento sigue indexado y recuperable a nivel documento. La limpieza posterior (normalizacion Unicode NFC, remocion de caracteres de control, deteccion de idioma) es deliberadamente conservadora sobre el contenido — sin lowercasing ni stemming — porque la evaluacion compara el campo *text* de forma textual contra el ground truth.

Reparacion del guion de fin de linea. Las fuentes incrustadas de muchos PDF no mapean el guion de corte a Unicode y el extractor lo entrega como U+FFFE, partiendo la palabra: *sate■lites*, *weap■ons*. Afectaba al **30% de los fragmentos** (38.397 de 128.526, en 613 documentos) y degradaba las dos metricas a la vez: rompe para el tokenizer justo los terminos de dominio que discriminan la consulta, y entrega al evaluador un fragmento con el texto partido. La reparacion decide entre unir (*satellites*) y conservar el guion (*AI-related*) contando en el propio corpus cual de las dos formas aparece mas veces — 23.177 cortes quedaron resueltos con esa evidencia — y solo cae en una heuristica cuando ninguna forma esta atestiguada. Medido sobre el ground truth propio, la reparacion no cambia F1@3 de forma significativa (0.306 a 0.310; gana 3 consultas, pierde 3, empata 35): se adopta porque corrige un defecto del dato y limpia los 166 fragmentos entregados que salian partidos, no por una ganancia de recuperacion que la

muestra no puede sostener.

1.1 Identificación de documentos (doc_id)

El emparejamiento a nivel documento se hace con el **DOC_ID que suministra ADL** en el inventario del corpus (formato *F1-AIINDEX-001*), no con un identificador propio. El manifest se indexa por **ruta relativa** y no por nombre de archivo: 59 nombres del inventario aparecen en dos carpetas distintas con DOC_ID distintos — los informes de CSET bajo *pdfs/Reports* y *pdfs/Translation*, y las teselas que se repiten por nivel de zoom — de modo que indexar por nombre le habría asignado el identificador equivocado a 127 documentos y anulado su aporte al F1@3. Once archivos presentes en el corpus no figuran en el inventario de ADL (catalogos y registros del proceso de descarga); al no tener DOC_ID no pueden aparecer en el ground truth, y se excluyen del índice para que no ocupen uno de los tres cupos de documento por consulta.

2. Estrategia de chunking y justificación

Se implemento una estrategia **hibrida estructural + oracional con solapamiento**, en tres niveles:

- **Estructural:** el texto se separa primero por encabezados Markdown (que HTML/JSON/MD ya traen marcados por el extractor); si el documento no tiene encabezados (caso típico de PDF), se trata como una única sección.
- **Segmentación de oraciones multilingüe:** dentro de cada sección, se usa *pysbd* (reglas lingüísticas dedicadas) para español e inglés, y el parser entrenado de spaCy (*pt_core_news_sm*) para portugués, que *pysbd* no cubre nativamente.
- **Empaquetado voraz por presupuesto de tokens** (~280 tokens, margen bajo el límite de 512 del encoder), contando tokens con el tokenizer real del encoder de indexación — no una aproximación por palabras — con **solapamiento de 1 oración** entre chunks consecutivos para no perder ideas que caen en la frontera.

La unidad mínima que se mueve entre chunks es siempre una oración completa: el empaquetador nunca corta un carácter a mitad de oración, cumpliendo el requisito de completitud lingüística (sec. 3.3). Esta garantía se verifica automáticamente en *tests/test_chunking.py*. La única excepción estructural es CSV/XLSX, donde cada fila (no una oración en lenguaje natural) se indexa como chunk atómico, tal como permite explícitamente la especificación.

3. Encoder(s) seleccionado(s) y criterios de elección

Encoder primario: **sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2** (arquitectura encoder tipo BERT, licencia Apache 2.0).

Criterio	Justificación
Soporte multilingüe	Cubre nativamente español, inglés y portugués (50+ idiomas), necesario porque el corpus y las consultas mezclan los tres.
Dimensionalidad	384 dimensiones: balance razonable entre costo de almacenamiento/busqueda y expresividad para el volumen esperado del reto.
Longitud máxima de entrada	512 tokens; el presupuesto de chunking (~280 tokens) deja margen suficiente.
Licencia	Apache 2.0, uso libre sin restricciones para el contexto del reto.
Eficiencia computacional	Modelo liviano (backbone MiniLM), adecuado para los recursos de cómputo limitados del equipo.
Riesgo de reproducibilidad	Ya validado en el material de apoyo entregado por el equipo organizador (01_embeddings.ipynb, 02_rag.ipynb), reduciendo el riesgo de incompatibilidades de último momento.

3.1 Encoder secundario y fusion de rankings (sec. 4.4)

Actualizacion: la entrega usa una cascada de **tres** encoders: MiniLM trae los candidatos y los re-puntuan **gte-multilingual-base** (encoder-only, Apache 2.0, 305 M, 768 dim) y E5, con peso 0,60 cada uno. Elegida midiendo cinco estructuras: NDCG@10 sube de **0,338 a 0,406** (41 consultas) y de 0,329 a 0,360 (10 independientes). GTE como *primario* se descarto pese a su mejor F1 en las 41 (0,385) porque cae a **0,200** en las independientes: sesgo de pooling.

El pipeline soporta construir la base con mas de un encoder. El secundario es **intfloat/multilingual-e5-base** (encoder tipo BERT, licencia MIT, 768 dimensiones, multilingue nativo, limite 512 tokens). El criterio de seleccion no fue elegir un modelo "mejor" en abstracto, sino uno con **errores decorrelacionados** respecto al primario: E5 esta entrenado para recuperacion densa mientras que el primario esta afinado para similitud de parafrasis. Fusionar dos rankings solo aporta cuando los modelos se equivocan en casos distintos; dos encoders de la misma familia habrian aportado poco.

La familia E5 exige prefijos distintos para consulta (*query:*) y pasaje (*passage:*); omitirlos degrada la calidad silenciosamente. Por eso la interfaz *Encoder* distingue codificacion asimetrica (*encode_query / encode_passages*), y los encoders que no los requieren heredan el comportamiento por defecto. Se verifico ademas que el bajo rendimiento del secundario en solitario es real y no un error de uso de esos prefijos: la indexacion aplica *passage:* y la consulta *query:*, tanto en el pipeline como en la copia autocontenida.

Los rankings no se fusionan simetricamente: los encoders secundarios **re-puntuan la lista del primario**, sumando su similitud a la del primario con un peso de 0,60 (los tres terminos son cosenos de la misma escala, de modo que sumarlos es legitimo). El RRF (ecuacion 7) queda en el codigo para la variante de varios primarios, que no es la entregada; fusionar listas distintas se midio y se rechazo (seccion 7), y el grafo no se incorpora como lista adicional sino como desempate (seccion 5).

Invariante de correccion: la fusion empareja fragmentos por *chunk_id*, lo que solo es valido si todos los indices comparten los mismos fragmentos. Como el presupuesto de chunking depende del tokenizer, fragmentar por separado con cada encoder produciria *chunk_id* colisionantes apuntando a textos distintos. Por eso el chunking se ejecuta una unica vez y sus fragmentos se reutilizan para todos los encoders; *generador.py* ademas verifica la coincidencia al cargar y aborta si detecta indices desincronizados, en lugar de producir resultados silenciosamente incorrectos.

Resultado de la medicion: la fusion simetrica no sirve, la cascada si. Comparadas contra el primario solo (F1@3 0,306 sobre las 41 consultas anotadas), *multilingual-e5-base* en solitario rinde 0,182 y pierde 7-17, la fusion RRF de ambos rinde 0,268 y pierde 8-13, y la **cascada 0,352 ganando 5-0 sin perder ninguna**. Sobre las 10 consultas de anotacion independiente el reparto es el mismo en direccion.

Re-medicion tras reparar los guiones (seccion 1). Reconstruidos los indices sobre el texto reparado la comparacion se sostiene: el primario solo pasa a **0,310 y 0,300**, y la cascada a **0,344 y 0,333**, ganando **4-0 y 1-0 sin perder ninguna**. La prueba de signos no alcanza significancia ($p = 0,125$); se conserva la cascada por el criterio fijado antes de medir, no por el promedio.

Por que la fusion simetrica no podia funcionar. RRF premia el *acuerdo* entre listas. Medido sobre las 50 consultas, los dos encoders comparten apenas el **11,3%** de los documentos del top-3 y el **6,2%** de los fragmentos del top-10. Con ese nivel de desacuerdo RRF no fusiona: intercala la lista buena con la mala, y el resultado queda a medio camino entre ambas. Eso es exactamente lo observado (0,306 baja a 0,268).

La cascada, que es lo que se entrega. Lo que E5 hace mal es el *recall*: su propio conjunto de candidatos rara vez contiene el documento correcto. Pero puntuar candidatos que el primario ya encontro es un trabajo distinto y mas facil. Asi que el primario genera 200 candidatos y E5 solo los **re-puntua**, sumando su similitud a la del primario con un peso de 0,60. Los dos terminos son cosenos, misma escala, de modo que sumarlos es legitimo. La operacion es barata porque los dos indices se construyen sobre los mismos fragmentos en el mismo orden: el vector de un fragmento en el segundo espacio se lee por su fila con *reconstruct*, sin volver a codificar ni un pasaje. El costo por consulta es una sola vectorizacion adicional.

El peso paso de 0,25 a **0,60**. El 0,25 se habia fijado cuando el pool era de 60 candidatos, la agregacion era *sum* y no habia glosario; las tres cosas cambiaron despues. Al ampliar el pool a 100 entran candidatos con similitud primaria mas baja, donde el mismo peso absoluto pesa mas en relacion a esa similitud, de modo

que el punto de equilibrio se desplaza. Se barrió la grilla completa 0,10 / 0,25 / 0,40 / 0,60 / 0,75 / 0,90 y el resultado es una **meseta, no una tendencia**: 0,60 es el único valor cuyo intervalo de confianza al 90% del delta pareado excluye una pérdida de 0,02 en las seis lecturas (F1@3 y NDCG@10 sobre las 50 consultas, sobre las 41 humanas y sobre las 10 de anotación independiente). Los valores 0,75 y 0,90 fallan el criterio sobre las 50. Que no haya tendencia monótona es lo que impide escalar el hallazgo a la hipótesis distinta de que el re-puntuador debería pasar a ser el primario.

No se emplea ningún modelo decoder/generativo en ninguna etapa de indexación o recuperación (sec. 4.2 y 8.3): en particular, se descarto explícitamente HyDE (requiere un LLM para generar una respuesta hipotética) y el reranking con LLM. El reranking de los candidatos lo resuelven dos encoders bi-encoder (gte y e5) que puntúan sobre los vectores ya indexados, sin releer texto. ADL confirmó que la restricción de la sec. 8.3 aplica a las arquitecturas decoder y que un *cross-encoder* (arquitectura encoder, no generativa) como re-puntuador está permitido; su evaluación se documenta en la sec. 7.

4. Índice FAISS

Se utiliza **IndexFlatIP** sobre vectores normalizados a norma unitaria, equivalente a similitud coseno (sec. 5.2 y 8.2). Es la opción recomendada por la propia especificación para el volumen de documentos esperado en esta etapa del reto: garantiza resultados exactos (sin la pérdida de exactitud de IVFFlat/HNSW) y su costo computacional es totalmente asumible a esta escala. El identificador interno de FAISS se mantiene alineado línea a línea con *metadata.jsonl* (mismo orden de inserción que de escritura), invariante verificado en *tests/test_faiss_alignment.py*. Si el corpus real de ADL resultara mucho mayor al esperado, *src/config.py* centraliza el punto donde migrar a un índice aproximado sin tocar el resto del pipeline.

La decisión se verificó empíricamente antes de fijarla. Con vectores de 384 dimensiones y $k=120$ sobre CPU, una búsqueda exacta tarda 1,9 ms (mediana) con 20.000 vectores, 5,7 ms con 50.000 (9,8 ms en el percentil 99) y 13,2 ms con 100.000. El corpus real produce **128.526 fragmentos**, es decir del orden de 15 ms por consulta: irrelevante frente a los ~27 ms que cuesta vectorizar la consulta y los ~19 s de carga del encoder. Migrar a IVFFlat o HNSW habría cambiado búsqueda exacta por búsqueda aproximada — perdiendo recall, que es justamente lo que mide la evaluación — a cambio de ahorrar milisegundos que nadie percibe.

5. Grafo de conocimiento (componente bonus)

NER: se usa el componente de reconocimiento de entidades ya entrenado en los modelos spaCy *es/en/pt_core_news_sm* (licencia MIT, arquitectura no generativa), filtrando tipos de entidad puramente numéricos/temporales (fechas, cantidades, porcentajes) para quedarse con personas, organizaciones, lugares y otras entidades nombradas relevantes.

Extracción de relaciones: heurística basada en dependencias sintácticas (permitida explícitamente por la especificación como alternativa a un modelo entrenado de RE): para cada oración con al menos dos entidades, se toma el verbo/auxiliar entre un par de entidades consecutivas como etiqueta de relación (o el verbo raíz de la oración si no hay uno intermedio), con una relación genérica de co-ocurrencia como último recurso.

Construcción e integración: las tripletas se acumulan en un *networkx.MultiDiGraph*, cada arista con *doc_id*, *chunk_id* y la relación como evidencia trazable, exportado a GraphML. En recuperación, las mismas entidades se detectan en la consulta, se buscan sus vecinos de primer orden en el grafo y se cuenta la evidencia de cada chunk. Como **integración no desplazante**: en lugar de fusionar los candidatos del grafo como una lista mas vía RRF (que medido degrada la recuperación), el grafo entra como clave secundaria de orden sobre los candidatos que el pool vectorial ya recuperó, rompiendo *solo* empates exactos de score por la evidencia de primer orden. El conjunto del pool no cambia, así que la integración no puede perjudicar la recuperación vectorial. Activa con el flag `--use-graph` de *generador.py*.

El grafo sobre el corpus real tiene 224.101 nodos y 754.876 aristas, construidas a partir de 1.687 documentos. Dos decisiones lo hicieron utilizable. La primera: se excluyen los formatos sin narrativa (CSV, XLSX y las teselas vectoriales, el 17,7% de los fragmentos), porque el NER busca entidades nombradas en

lenguaje natural y aplicado a una fila de tabla produce ruido — las entidades mas frecuentes de una version preliminar eran *FALSO*, *VERDADEIRO* y el nombre de una capa de mapa. La segunda: los nombres de entidad se limpian de caracteres de control, que el texto de OCR arrastra y que hacen fallar la exportacion a GraphML, un formato XML que no los admite.

La fusion plena del grafo esta rechazada por medicion; la integracion adoptada es la no desplazante. Con el ground truth propio (50 consultas), fusionar los candidatos del grafo con el ranking vectorial por RRF degrada la recuperacion: $F1@3$ 0,358 contra 0,455 y $NDCG@10$ 0,390 contra 0,516, con 18 consultas en cero. En cambio, el grafo como clave secundaria de orden (seccion 5) es **neutro sobre las 50 consultas**: 34 de ellas generan evidencia de grafo (1.029 apuntes de evidencia en total), pero al no existir empates exactos de score en el pool, ninguna linea de *resultados.jsonl* cambia frente a la corrida sin grafo. Es decir: el componente bonus queda integrado a la recuperacion, como exige la seccion 7, sin renunciar a ninguna fraccion de la metrica.

6. Modulo de recuperacion

- Mismo encoder para indexacion y consulta; vector de consulta normalizado igual que el indice.
- Sobre-recuperacion de candidatos antes de aplicar post-filtros de metadata (fenomeno, formato, idioma) o de umbral de similitud coseno (sec. 8.7).
- Agregacion a nivel documento por **suma** de las puntuaciones de sus fragmentos, sumando los **5 mejores** de cada uno, sobre un pool de **100 candidatos** mas amplio que los 10 fragmentos mostrados, para que la relevancia de un documento no dependa solo de si su mejor chunk aparece en el top-10 (sec. 8.6). La eleccion de la suma sobre el maximo esta medida, no supuesta: ver seccion 7.
- Combinacion de encoders por **suma ponderada de similitudes** sobre una sola lista (la cascada, sec. 3.1): cada re-puntuador anade su coseno multiplicado por el peso 0,60; RRF ($k=60$) queda disponible solo cuando se piden varios encoders primarios, que no es la configuracion entregada. El grafo no se fusiona por RRF (medido, degrada) sino que participa como clave secundaria de orden, ver seccion 5.
- **Post-filtrado por fenomeno dominante** (sec. 8.7): cuando un solo fenomeno acapara al menos el 80% del pool agregado a documento, el resultado se restringe a ese fenomeno. La sec. 10.2.2 empareja por *doc_id* y los documentos relevantes de casi todas las consultas viven en un unico fenomeno, asi que filtrar el 20% de candidatos ajenos concentra los cupos sin riesgo de vaciar la respuesta (medido: $F1@3$ 0,440 a 0,455). El umbral 0,8 se eligio por robustez, no como argmax.
- Limite de 250 palabras por fragmento aplicado dividiendo en limites oracionales completos cuando un chunk lo excede; los sub-fragmentos comparten *chunk_id* y ocupan su propio *rank* (sec. 9.2.1). El corpus real obligo a agregar un ultimo recurso: en texto proveniente de OCR la puntuacion puede desaparecer por completo y el segmentador devuelve una unica "oracion" de mas de 250 palabras, que respetar el limite oracional dejaria pasar entera. En ese caso — y solo en ese — el corte se hace por palabras.
- **Supresion de fragmentos con texto repetido.** El corpus contiene documentos duplicados (el mismo informe de CSIS bajo dos *doc_id*, series que reeditan capitulos completos), de modo que dos fragmentos distintos del indice pueden tener texto identico. Entregar el mismo texto dos veces no puede aportar ganancia en $NDCG@10$ — el ranking ideal no lo contiene — y ademas desplaza fuera del top-10 a un candidato que si podria aportarla. Sobre las 50 consultas oficiales, 17 de los 500 fragmentos entregados eran duplicados exactos; suprimirlos y completar el cupo con el siguiente candidato recupera esos 17 espacios. La comparacion es de texto exacto (normalizando solo espacios y mayusculas) y no de similitud aproximada, que podria descartar fragmentos legitimamente distintos que comparten un parrafo.
- **Los fragmentos se ordenan despues de decidir los documentos**, con tres criterios de desempate y sin descartar nada, para que las dos mitades de la respuesta no se contradigan: primero los fragmentos de los 3 documentos entregados; luego los que el evaluador puede leer (el corpus trae traducciones al coreano, ruso, arabe, chino y aleman que competian por los mismos cupos); por ultimo los que son aparato bibliografico, que la sec. 10.2.1 juzga por su texto y puntuan 0 aunque su documento sea el correcto. El primer criterio es la mejora individual mas grande del proyecto: los fragmentos procedentes

del top-3 pasan del **32% al 98%** y el NDCG@10 sube **+0,131** (IC 90% [+0,066, +0,199]) sin mover el F1@3, como corresponde a un cambio que no toca los documentos.

Ninguna etapa de recuperacion usa un modelo generativo: solo vectores, puntuaciones de similitud coseno y aritmetica sobre metadata.

7. Como se tomaron las decisiones: medicion interna

El ground truth oficial no es publico durante el reto, de modo que ninguna decision de diseno puede validarse contra la metrica real. Para no elegir a ojo se construyo uno propio (*dev/eval/*) que cubre **las 50 consultas oficiales**: 41 con documentos relevantes marcados y 9 en las que se reviso candidato por candidato sin que ninguno respondiera. Se construyo en dos etapas y la distincion es importante para no enganarse:

- **10 consultas de anotacion independiente.** Los candidatos salieron de contar palabras clave sobre el texto extraido, *sin pasar por el recuperador*. Son las unicas validas para comparar dos encoders entre si.
- **31 consultas por pooling** (la tecnica de TREC): los candidatos lo propuso el propio sistema. Aportan volumen, pero una consulta cuyos candidatos propuso el encoder X favorece a X, porque un documento que X nunca recupero no pudo marcarse. Por eso cada linea registra su procedencia y la evaluacion tiene una opcion *--sin-pooling*.

El sesgo resulto menor de lo temido: el F1@3 da 0,306 sobre las 41 consultas y 0,300 sobre las 10 independientes, porque los candidatos se proponen con un pool de 200 mientras la configuracion entregada agrega 100: las marcas no se cumplen solas.

La agregacion a documento **suma** las puntuaciones de sus fragmentos en lugar de tomar el **maximo**, y con el pool ampliado a 100 suma solo los 5 mejores. La razon principal no es el promedio (0,306 frente a 0,226 sobre las 41 anotadas, pero contando por consulta 16-8 con 17 empates, **p = 0,15**: sin significancia) sino el argumento estructural: un documento relevante contiene *varios* pasajes relevantes, mientras que el maximo premia al que tuvo un unico fragmento afortunado. Se documenta asi y no como resultado medido; la diferencia entre "lo medimos" y "lo argumentamos y el dato no lo contradice" es lo que evita sobreajustar a un ground truth reducido.

El promedio de F1@3 no basta para decidir. Con 26 consultas el pool de 30 parecia superior a 60 (0,309 frente a 0,274), pero contando por consulta el reparto es **5-3 con 18 empates**, lo que produce el azar. El pool quedo en 60 hasta que una medicion posterior, con criterio de intervalo de confianza, justifico ampliarlo a 100.

El mismo criterio descarto dos hipotesis prometedoras: la recuperacion entre idiomas distintos (2 de 5 aciertos con documentos en ingles frente a 3 de 5 en espanol) y la confusion entre grupos armados ilegales y fuerzas armadas estatales (medido con un patron justo, afecta al 8% de los cupos). El riesgo de un ground truth reducido no es medir de menos sino *sobreajustar*.

Donde esta el cuello de botella. Antes de seguir ajustando parametros se midio que impide acertar cuando el sistema falla, separando los tres motivos de un F1@3 nulo: documento no indexado, ningun fragmento suyo en el pool, o que entre y pierda la agregacion. De las 17 consultas en cero, ninguna por indexacion, dos porque el encoder no alcanza el documento y **quince con el documento correcto dentro del pool**. El limite no esta en la recuperacion sino en la agregacion.

De ese diagnostico salieron dos intentos, ambos implementados y descartados por la misma regla. El primero, sumar solo los *M* mejores fragmentos de cada documento: la suma sin tope deja que un documento con muchos fragmentos mediocres desplace a uno con un fragmento excelente. El promedio mejora (0,347 con pool 100), pero es el maximo de setenta combinaciones y por consulta reparte 8-3 con 30 empates (**p = 0,227**). El segundo, BM25 fusionado con el denso por RRF para rescatar los terminos raros que un vector de 384 dimensiones diluye: rinde 0,192 frente a 0,306 y pierde 15-4 (**p = 0,019**). La hipotesis lexica queda refutada como mejora general.

La ultima mejora adoptada es el post-filtrado por fenomeno dominante (sec. 8.7, detallado en la seccion 6): cuando un solo fenomeno ocupa al menos el 80% del pool agregado, el resultado se restringe a

el. El umbral 0,8 no es el argmax — 0,7 da mejor F1@3 sobre las 50 — sino el unico que no dispara el veto pre-registrado: por debajo, las consultas en cero suben de 11 a 12, y el filtro no toca las consultas con relevantes en dos fenomenos a la vez (q019). Cambia 6 de 50 lineas y lleva la entrega a su **estado final**: F1@3 = 0,455 y NDCG@10 = 0,516 (0,499 penalizado), con 11 consultas en cero. El F1@3 no tiende a 1: la sec. 10.2.2 fija P@3 = aciertos/3 con los tres cupos siempre llenos, asi que una consulta con un solo documento relevante topa en 0,50 y el techo sobre este ground truth es **0,906** — el 0,455 es el 50% de lo alcanzable, no el 45% de 1.

Los dos ultimos cambios ordenan los fragmentos por lo que la sec. 10.2.1 juzga: su campo text.

Un fragmento ilegible, o que no menciona el objeto de la consulta, vale cero por relevante que sea su documento. El **idioma legible sube por encima de la alineacion con el top-3** (antes un fragmento en coreano del documento n.º 1 desplazaba a uno legible del n.º 4: ilegibles **19 a 0**) y la **cobertura lexica de la consulta** entra como ultimo desempate (fragmentos sin una palabra de la consulta, **175 a 122** de 500). Juntos, NDCG@10 **+0,016** (IC 90% [+0,004, +0,029], 11-4) y F1@3 inalterado.

Las dos ultimas mejoras adoptadas se componen. La primera es **ampliar el pool de 60 a 100**: la cascada recuperaba 200 candidatos y se descartaban 140 antes de agregar, de modo que la profundidad extra solo reordenaba. La segunda es un **glosario bilingüe** que expande la consulta antes de vectorizarla, justificado por una propiedad medible del corpus: las consultas estan en espanol y los fenomenos 1 y 2 en ingles, y *todos* los terminos de dominio que las consultas usan en espanol son entre 30 y 2.000 veces mas raros que su forma inglesa (NBQR 0 veces contra 66 de CBRN; "antisatelite" 8 contra 3.813 de ASAT). Es una tabla escrita a mano, sin modelo generativo ni traductor (sec. 8.3); una consulta sin terminos de la tabla se devuelve identica.

Medidas sobre el ground truth propio, cada una por separado y las dos juntas (F1@3 y NDCG@10, primero sobre las 50 consultas y luego sobre las 41 de anotacion humana): la configuracion anterior daba **0,363 / 0,392 y 0,386 / 0,405**; solo el glosario, 0,379 / 0,403 y 0,396 / 0,406; solo el pool ampliado, 0,366 / 0,434 y 0,398 / 0,447; y **las dos juntas, que son las que se entregan, 0,402 / 0,457 y 0,424 / 0,456**. Los efectos se componen: ninguna de las dos anula a la otra.

La ultima ronda anadio dos palancas mas, y tambien se componen. El peso del re-puntuador paso de 0,25 a 0,60 (sec. 3.1) y el glosario gana tres entradas — *derecho internacional en el espacio, dominio espacial y sistemas no tripulados* — medidas una por una, cada una ganando una consulta sin perder ninguna. Que se compusieran no era obvio: el glosario cambia *que* candidatos entran al pool y el peso reordena el pool ya recuperado, de modo que un re-puntuador con mas peso podia hundir justo los documentos que el glosario acababa de rescatar. Medido, no ocurre: partiendo de 0,402 / 0,457 sobre las 50, solo el peso da 0,425 / 0,476, solo el glosario 0,423 / 0,486, y **las dos juntas 0,440 / 0,490**, mejor que cualquiera por separado en las cuatro lecturas comparadas.

La salvedad, que se declara junto al numero: contra el glosario solo, la ganancia sobre las 50 consultas es estrecha (NDCG@10 +0,004). Lo que sostiene la combinacion es el F1@3 sobre las 10 consultas de anotacion independiente, +0,067 con intervalo al 90% de [+0,000, +0,133] — la unica muestra libre del sesgo de pooling, y tambien la mas pequena: una sola consulta la mueve 0,033. Sobre las 50, el cambio gana 9 consultas y *pierde* 4.

El glosario dejo ademas un hallazgo que acota su propio alcance: **la asimetria espanol/ingles existe solo en los fenomenos 1 y 2**. En el fenomeno 3, territorial y colombiano, la relacion se invierte ("reclutamiento" en 682 fragmentos contra 2 de *child recruitment*), de modo que expandir al ingles una consulta de ese fenomeno la alejaria de sus documentos: es una herramienta de dos fenomenos, no de tres. Por la misma razon se rechazo *capacidades laser* → *laser weapons*: "laser" es cognado y la consulta ya tenia puente al corpus ingles.

Con el pool ampliado la agregacion pasa de *suma* a **sumar los 5 mejores fragmentos** de cada documento, por robustez: con pool 60 las dos son identicas, pero al ampliarlo la suma pierde el tope y un documento con muchos fragmentos mediocres desplaza al bueno — con pool 200 cae a **0,298** mientras el tope de 5 sube a 0,406.

Lo que cuestan, dicho explicitamente: sobre las 10 consultas de anotacion independiente el F1@3 baja de 0,333 a 0,300 y el NDCG@10 de 0,360 a 0,338. Es *una sola consulta* la que cambia de lado y el intervalo de confianza con esa muestra no distingue la perdida de cero, pero es la unica muestra sin sesgo

de pooling y corresponde decir que ahí no confirman. Se adoptan porque ganan de forma consistente en las 41 de anotación humana — $F1@3 +0,038$ (IC 90% [$+0,002$, $+0,073$]) y $NDCG@10 +0,051$ ([$+0,008$, $+0,095$]) — y porque las dos tienen explicación mecánica previa a la medición. Dos ajustes se midieron y no se adoptaron: reservar cupos de fragmento degrada el $NDCG@10$ de forma monótona ($-0,025$ con 8, $-0,082$ con 4) y el prior de recencia es inerte; quedan apagados. El grafo fusionado pierde 11-0 (sección 5); como clave secundaria no desplaza nada y se adopta sin tocar métricas.

El cross-encoder se evalúa y se descarta. ADL confirmó que la sec. 8.3 aplica a arquitecturas decoder, así que un *cross-encoder* (encoder, no generativo) como re-puntuador está permitido. Se probó *bge-reranker-v2-m3* sobre el pool entregado, con el arnés verificado contra *resultados.jsonl* (50/50 idénticas). La mejor celda sobre las 50 da $F1@3 0,465$, pero en las 10 independientes cae de $0,433$ a $0,367$ (IC [$-0,133$, $0,000$]): ganancia de pooling, la firma que descarta a las hipótesis anteriores; y sumarla a gte y e5 sube las consultas en cero de 11 a 12. Se descarta por el criterio vigente y por costo: ~85 s/consulta en CPU contra milisegundos de la cascada, que lee vectores sin releer texto.

La entrega se verifica de punta a punta antes de empaquetarse con *scripts/validar_entrega.py*, que comprueba la estructura de carpetas, las 50 líneas, los 3 documentos y 10 fragmentos por consulta, el límite de 250 palabras, la alineación entre el índice FAISS y *metadata.jsonl*, y que todo *doc_id* reportado pertenezca al inventario de ADL. La suite de pruebas (161 casos) cubre los invariantes críticos, incluida la ejecución de *generador.py* como subprocesso con *PYTHONPATH* vacío para garantizar que es autocontenido y que los resultados son reproducibles, requisito del punto 4 de la sec. 1.4.

8. Limitaciones conocidas y decisiones documentadas

- **El ground truth propio es la limitación dominante del proyecto.** Los valores de $F1@3$ de la sección 7 sirven para ordenar configuraciones entre sí, no para estimar la nota: la anotación es parcial frente a un corpus de 1826 documentos, así que un documento relevante no anotado cuenta como error y el valor real será mayor. Además se juzga el documento viendo un solo fragmento — el mejor de cada documento, que no siempre es la mejor evidencia — lo que hace marcar de menos, nunca de más.
- **El $NDCG@10$ se mide con un proxy:** la relevancia de cada fragmento se hereda de su documento, porque anotarla de verdad exigiría relevancia graduada fragmento por fragmento. El proxy sobreestima — da 1 a la bibliografía de un documento relevante — y cuanto, está acotado por abajo: descontando el aparato bibliográfico da 0,499 frente a 0,516. El otro modo de fallo, un pasaje que no responde dentro de un documento que sí es relevante, no lo ve ninguna medición automática.
- **Documentos sin texto recuperable:** 8 de los 1826. Las cinco imágenes no aportan texto (una en AVIF, que Pillow no lee sin plugin) pero quedan presentes en *metadata.jsonl* con una fila sin vector (*en_indice* = false), manteniendo la alineación índice–metadata completa. Los otros tres — un JSON de 0 bytes y dos HTML de error con extensión .pdf, detectados por contenido — no aportan texto y quedan sin posibilidad de ser recuperados.
- **Campo fuente:** se reporta el nombre de archivo estandarizado del inventario de ADL, nunca la URL (que se conserva en un campo *url* adicional). La sec. 10.2.1 sugería *fuentes* como clave de emparejamiento y ADL aclaró que es *doc_id*; el nombre de archivo funciona bajo cualquiera de las dos lecturas.
- **Fusión de chunks adyacentes** (sec. 9.2.1, opcional): implementada y revertida — el chunker solapa una oración y concatenar duplica texto.
- **Boilerplate de PDF:** el filtro de líneas repetidas pide 3 o más páginas.